

# TheKnife - Manuale Tecnico

## Indice

1. Introduzione e Architettura
  2. Diagrammi UML e Modello ER
  3. Struttura del Progetto (Maven Multi-Module)
  4. Persistenza dei Dati (PostgreSQL e DAO)
  5. Protocollo di Rete (Network)
  6. Modulo Server (Concorrenza e Socket)
  7. Modulo Client (JavaFX e GUI)
  8. Sicurezza
  9. Flusso di Esecuzione
  10. Compilazione, Avvio e Packaging
  11. Riferimenti e Fonti
- 

## 1. Introduzione e Architettura

TheKnife è una piattaforma di scoperta e recensione ristoranti ispirata a TheFork. Il sistema è stato sviluppato in Java 17 utilizzando un'architettura distribuita **Multi-Tier e Multi-Modulo**, gestita tramite Apache Maven.

L'architettura adottata separa nettamente le responsabilità in tre moduli principali:

- **common** : Il core del sistema. Contiene il modello dati (POJO), i Data Access Object (DAO) per l'interazione con PostgreSQL, la logica di business (Gestori) e le classi del protocollo di rete condivise.
- **server** : Espone i servizi via rete attraverso **Socket TCP**. Gestisce le richieste concorrenti dei client istanziando un Thread per ogni connessione (pattern Thread-per-Request).
- **client** : Fornisce l'interfaccia utente grafica (GUI) sviluppata in **JavaFX**. Si occupa di raccogliere l'input, serializzarlo e inviarlo al server.

## Pattern di Progettazione Utilizzati

- **DAO (Data Access Object):** Isola completamente l'applicazione dal DBMS. I Gestori non scrivono SQL, ma delegano ai DAO.
  - **MVC (Model-View-Controller):** Model (classi in `model/` ), View (classi JavaFX in `client/gui/` ), Controller (Gestori in `gestione/` e `ClientHandler` nel server).
  - **Observer/Listener (JavaFX):** La GUI utilizza i `Task` di JavaFX per gestire le chiamate di rete asincrone su thread separati, evitando il blocco dell'interfaccia utente (Application Thread).
- 

## 2. Diagrammi UML e Modello ER

### 2.1 Diagramma dei Casi d'Uso (UML)

Il diagramma dei casi d'uso mappa le interazioni tra gli attori (Ospite, Cliente, Ristoratore) e il sistema. L'Ospite può registrarsi, fare login e cercare ristoranti. Il Cliente eredita le funzionalità dell'Ospite e può aggiungere preferiti e gestire recensioni. Il Ristoratore può aggiungere ristoranti e rispondere alle recensioni.

### 2.2 Modello Entity-Relationship (ER)

Il database si compone di tre entità principali (Utenti, Ristoranti, Recensioni) e di un'associazione M:N (Preferiti).

- **Utenti** (ID, Nome, Cognome, Username, Password\_Hash, Ruolo, ecc.)
- **Ristoranti** (ID, Nome, Città, Indirizzo, Latitudine, Longitudine, Fascia\_Prezzo, ecc.)
- **Recensioni** (ID, Stelle, Testo, Data\_Creazione, Risposta, ecc.)

Le relazioni sono:

- **Gestisce** (Utente 1 - N Ristoranti)
- **Recensisce** (Utente N - N Ristoranti, con attributi)
- **Preferisce** (Utente N - N Ristoranti, tabella ponte)

### 2.3 Diagramma delle Classi (UML)

Il diagramma delle classi illustra l'architettura software divisa per stereotipi ( «Model» , «DAO» , «Controller» , «Network» , «Server» , «Client» ). Mostra le relazioni di dipendenza tra i Gestori e i DAO, e le associazioni tra i DAO e le classi Modello.

---

## 3. Struttura del Progetto (Maven Multi-Module)

La struttura delle cartelle riflette la separazione dei moduli Maven:

```
THE-KNIFE/
  pom.xml          # POM padre (packaging: pom)
  common/          # Modulo Core (Logica e Dati condivisi)
    pom.xml        # Dipendenze: PostgreSQL JDBC
    src/main/java/theknife/
      dao/          # Data Access Object (Interazione DB)
      gestione/     # Business Logic Layer
      model/         # Data Model (POJO)
      network/       # Protocollo di comunicazione (Request, Response)
      util/          # Utility (Hashing SHA-256, Import CSV)
  server/          # Modulo Server (Socket TCP e Concorrenza)
    pom.xml        # Dipendenze: maven-shade-plugin
    src/main/java/theknife/server/
      ServerMain.java # Entry point del server (porta 5000)
      ClientHandler.java # Gestione thread per ogni client
  client/          # Modulo Client (Interfaccia Grafica JavaFX)
    pom.xml        # Dipendenze: JavaFX, maven-shade-plugin
    src/main/java/theknife/client/
      ServerConnection.java # Gestione socket lato client
      SessionManager.java  # Stato sessione utente
      gui/                  # Schermate JavaFX (Login, Register, Search, ecc.)
  bin/               # File eseguibili .jar per la consegna
  db/               # Script di creazione database (schema.sql)
  doc/              # Manuale Utente, Manuale Tecnico, Diagrammi, Javadoc
```

---

## 4. Persistenza dei Dati (PostgreSQL e DAO)

Il sistema abbandona la gestione basata su file per abbracciare un vero DBMS relazionale (PostgreSQL 16). Questo garantisce ACIDità, concorrenza sicura ed efficienza nelle ricerche.

### 4.1 Il Pattern DAO

Le classi nel package `dao` isolano il codice SQL. Utilizzano esclusivamente `PreparedStatement` per prevenire attacchi SQL Injection.

La classe `DatabaseConnection` gestisce il pool di connessioni leggendo le credenziali da un file esterno `db.properties.env`.

## 4.2 Query Avanzate e Viste SQL

Il `RistoranteDAO` implementa un metodo `cercaConCriteri` che costruisce dinamicamente una query SQL tramite `StringBuilder` in base ai filtri scelti dall'utente (cucina, prezzo, delivery). Per ottimizzare le performance, la query effettua una `LEFT JOIN` con la vista SQL `vista_valutazioni_ristoranti`, calcolando la media delle stelle e il numero di recensioni direttamente a livello di database, evitando inutili caricamenti in memoria.

## 4.3 Gestione Transazionale e Vincoli

Il database impone vincoli di integrità referenziale ( `FOREIGN KEY` ) e regole di dominio:

- `UNIQUE (nome, indirizzo)` sui ristoranti per evitare duplicati durante l'importazione massiva del dataset Michelin.
  - `UNIQUE (id_utente, id_ristorante)` sulle recensioni per impedire a un utente di recensire più volte lo stesso locale (gestito nel DAO intercettando l'errore SQL 23505 ).
- 

## 5. Protocollo di Rete (Network)

La comunicazione tra Client e Server avviene tramite serializzazione Java su Socket TCP. Il protocollo è basato su scambio di oggetti (POJO serializzabili):

- **Request** : Contiene un enum `RequestType` (es. LOGIN, SEARCH, ADD\_REVIEW) e una `Map<String, Object>` per i parametri.
  - **Response** : Contiene un boolean `success`, un `message` testuale e un campo `Object data` per trasportare il risultato (es. l'oggetto `Utente` loggato o una `List<Ristorante>` ).
  - **SearchCriteria** : Oggetto serializzabile utilizzato per incapsulare i filtri di ricerca avanzata.
- 

## 6. Modulo Server (Concorrenza e Socket)

Il server è il cuore del sistema e gestisce la logica in modo concorrente.

- **ServerMain** : Apre una `ServerSocket` sulla porta 5000. In un ciclo `while(true)` accetta le connessioni. Per ogni `Socket` accettato, crea un `new Thread(new ClientHandler(socket)).start()`, garantendo che più client possano operare in parallelo senza bloccarsi a vicenda.

- **ClientHandler** : Nel metodo `run()` , legge l'oggetto `Request` tramite `ObjectInputStream` . Utilizza uno `switch` sul `RequestType` per smistare la richiesta. Istanza i Gestori (che a loro volta usano i DAO per interrogare PostgreSQL), elabora la logica, impacchetta il risultato in un oggetto `Response` e lo rimanda tramite `ObjectOutputStream` . Alla fine, chiude la connessione.
- 

## 7. Modulo Client (JavaFX e GUI)

L'interfaccia grafica è sviluppata in JavaFX e segue il pattern MVC.

- **Launcher / TheKnifeApp** : Classi di avvio della GUI. `Launcher` serve come classe principale per l'esecuzione del file `.jar` , aggirando le restrizioni del modulo JavaFX che impedirebbero l'avvio diretto di una `Application` in un Fat Jar.
  - **SceneManager** : Classe statica che gestisce il cambio delle scene (schermate). Mantiene il riferimento alla `ServerConnection` .
  - **ServerConnection** : Incapsula la logica Socket. Il metodo `sendRequest(Request)` apre la connessione, invia l'oggetto, aspetta la `Response` e la ritorna.
  - **SessionManager** : Mantiene lo stato dell'utente loggato lato client.
  - **View (LoginView, RegisterView, SearchView, ecc.)**: Classi che costruiscono l'interfaccia UI. Per evitare il blocco della UI durante le chiamate di rete (che sono bloccanti), utilizzano `Task<Response>` avviati su `Thread` separati (impostati come daemon thread). Al completamento del Task, il risultato viene riassunto sul JavaFX Application Thread per aggiornare la UI.
- 

## 8. Sicurezza

La gestione della sicurezza è affidata alla classe `PasswordUtil` .

- **Hashing Password**: Le password non vengono mai salvate in chiaro. Prima di essere inviate al database, vengono cifrate utilizzando l'algoritmo **SHA-256** (Secure Hash Algorithm 256-bit), che genera un hash esadecimale a 64 caratteri.
  - **Domande di Sicurezza**: Anche le risposte alle domande di sicurezza per il recupero della password vengono sottoposte a hashing prima del salvataggio.
  - **Credenziali DB**: Le credenziali di accesso al database PostgreSQL sono memorizzate in un file `db.properties.env` esterno al codice, ignorato da Git (tramite `.gitignore` ), per prevenire fughe di dati sensibili nei repository pubblici.
- 

## 9. Flusso di Esecuzione

Di seguito è illustrato il flusso di una tipica richiesta di ricerca ristoranti:

1. L'utente compilare i campi nella `SearchView` (JavaFX) e clicca "Cerca".
  2. La View crea un oggetto `Request` di tipo `SEARCH`, inserendo i criteri in un oggetto `SearchCriteria`.
  3. Un `Task` di JavaFX invia la `Request` al server tramite `ServerConnection`.
  4. Il `ServerMain` accetta la connessione e passa la `Request` a un `ClientHandler` (su un Thread separato).
  5. Il `ClientHandler` legge la `Request`, estrae i criteri e chiama `GestoreRistoranti.cercaRistorante()`.
  6. Il `GestoreRistoranti` delega al `RistoranteDAO.cercaConCriteri()`, che esegue la query SQL su PostgreSQL.
  7. PostgreSQL restituisce un `ResultSet`, che il DAO converte in una `List<Ristorante>`.
  8. Il `ClientHandler` impacchetta la lista in un oggetto `Response` e la rimanda al Client.
  9. Il `Task` di JavaFX riceve la `Response`, estrae la lista e aggiorna la `TableView` a schermo.
- 

## 10. Compilazione, Avvio e Packaging

Il progetto è gestito da Apache Maven. Sono disponibili due modalità di avvio: tramite codice sorgente (Maven) o tramite file eseguibili `.jar` precompilati.

### 10.1 Setup del Database

1. Eseguire lo script `db/schema.sql` su PostgreSQL.
2. Configurare le credenziali nel file `db.properties.env` nella root del progetto.

### 10.2 Avvio tramite Maven (Sviluppo)

Per compilare e avviare l'applicazione direttamente dal codice sorgente:

1. **Compilazione totale:**

```
mvn clean install
```

2. **Avvio del Server:**

```
mvn exec:java -pl server "-Dexec.mainClass=theknife.server.ServerMain"
```

### 3. Avvio del Client (GUI) su un nuovo terminale:

```
mvn javafx:run -pl client
```

## 10.3 Generazione ed Avvio tramite File JAR (Produzione)

Per generare i file `.jar` eseguibili (contenenti tutte le dipendenze, creati tramite `maven-shade-plugin`):

### 1. Packaging:

```
mvn clean package
```

2. I file `.jar` verranno generati nelle cartelle `server/target/` e `client/target/`.  
Posizionarli in una cartella `bin/`.

### 3. Avvio del Server:

```
java -jar bin/serverTK.jar
```

### 4. Avvio del Client (GUI) su un nuovo terminale:

```
java -jar bin/clientTK.jar
```

---

## 11. Riferimenti e Fonti

La progettazione e lo sviluppo del progetto si sono basati sulle seguenti risorse ufficiali e documentazioni tecniche:

- **Oracle Java Documentation:** Documentazione ufficiale del linguaggio Java SE 17.
  - *Link:* <https://docs.oracle.com/en/java/javase/17/>
- **PostgreSQL Official Documentation:** Manuale ufficiale per l'implementazione del database relazionale.
  - *Link:* <https://www.postgresql.org/docs/>
- **Apache Maven Documentation:** Guida ufficiale per la gestione delle dipendenze e configurazione Multi-Modulo.
  - *Link:* <https://maven.apache.org/guides/>

- **JavaFX Documentation:** Documentazione ufficiale per lo sviluppo dell'interfaccia grafica (GUI).
    - *Link:* <https://openjfx.io/javadoc/17/>
  - **JDBC API Documentation:** Specifiche per la connessione a database tramite Java.
    - *Link:* <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>
  - **SHA-256 Algorithm:** Specifiche e dettagli sull'algoritmo di hashing crittografico utilizzato per la sicurezza delle password.
    - *Link:* <https://en.wikipedia.org/wiki/SHA-2>
  - **Kaggle Dataset (Michelin Guide):** Fonte del dataset `michelin_my_maps.csv` utilizzato per l'importazione massiva dei ristoranti.
    - *Link:* <https://www.kaggle.com/datasets/ngshiheng/michelin-guide-restaurants-2021>
  - **Specifiche di Progetto:** Regolamento e linee guida del corso di Laboratorio Interdisciplinare B, Università degli Studi dell'Insubria (a.a. 2024/2025).
- 

**Fine Manuale Tecnico**